

UNIT 2

Deep Learning Foundations for Generative AI

Detailed Study Notes

Neural Network Architectures: Feedforward, CNN & RNN | Review of Autoencoders: Vanilla & Sparse | Variational Autoencoders: Latent Variables & Reconstruction | Loss Functions: Cross-Entropy, KL Divergence, Reconstruction Loss

Prepared for classroom and self-study use. Covers every syllabus sub-topic with definitions, formulas, worked explanations, comparison tables and exam-style key points.

Table of Contents

1. Neural Network Architectures: Feedforward, Convolutional, and Recurrent Networks
 - 1.1 Feedforward Neural Networks (FNNs / MLPs)
 - 1.2 Convolutional Neural Networks (CNNs)
 - 1.3 Recurrent Neural Networks (RNNs) and Variants (LSTM, GRU)
 - 1.4 Comparative Summary of Architectures
 2. Review of Autoencoders: Vanilla and Sparse Autoencoders
 - 2.1 What Is an Autoencoder?
 - 2.2 Vanilla (Standard) Autoencoder
 - 2.3 Sparse Autoencoder
 - 2.4 Other Autoencoder Variants (Denoising, Contractive, Undercomplete/Overcomplete)
 3. Variational Autoencoders (VAEs): Latent Variable Representation and Reconstruction
 - 3.1 From Deterministic to Probabilistic Autoencoders
 - 3.2 Latent Variable Representation
 - 3.3 The Reparameterization Trick
 - 3.4 Reconstruction in VAEs
 - 3.5 VAE vs Vanilla Autoencoder
 4. Loss Functions: Cross-Entropy, KL Divergence, and Reconstruction Loss
 - 4.1 Cross-Entropy Loss
 - 4.2 KL Divergence
 - 4.3 Reconstruction Loss
 - 4.4 Putting It Together — The VAE Total Loss (ELBO)
- Unit 2 — Quick Revision Summary

1. Neural Network Architectures: Feedforward, Convolutional, and Recurrent Networks

Before studying autoencoders and VAEs, it is essential to understand the three foundational neural network architectures that serve as the building blocks (especially as encoders/decoders) inside almost every generative model.

1.1 Feedforward Neural Networks (FNNs / MLPs)

DEFINITION — Feedforward Neural Network

The simplest type of artificial neural network, also called a **Multi-Layer Perceptron (MLP)**, in which information flows in only **one direction** — from the input layer, through one or more hidden layers, to the output layer — with no cycles or loops/feedback connections.

Structure and components:

- **Input Layer:** Receives the raw feature vector (e.g., pixel values flattened into a vector, or tabular features). Number of nodes = number of input features.
- **Hidden Layer(s):** One or more layers of neurons that transform the input using learned weights, biases, and non-linear activation functions.
- **Output Layer:** Produces the final prediction (e.g., class probabilities via Softmax for classification, a continuous value for regression).
- **Fully Connected (Dense) Layers:** Every neuron in one layer is connected to every neuron in the next layer.

Mathematical operation at a single layer:

$$h = f(W \cdot x + b)$$

where x is the input vector, W is the weight matrix, b is the bias vector, f is a non-linear activation function (e.g., ReLU, Sigmoid, Tanh), and h is the layer's output (activation).

Training mechanism:

- **Forward Propagation:** input passes layer by layer to produce an output/prediction.
- **Loss Computation:** the prediction is compared to the true target using a loss function.
- **Backpropagation:** gradients of the loss with respect to every weight are computed using the chain rule, propagating error backward from output to input layer.
- **Weight Update:** an optimizer (SGD, Adam, RMSProp) updates weights in the direction that reduces the loss.

Key characteristics and limitations:

- **Universal Approximation Theorem:** a feedforward network with at least one hidden layer and enough neurons can, in theory, approximate any continuous function to arbitrary accuracy.
- **No memory of order/position:** FNNs treat input features independently and have no built-in notion of spatial locality (unlike CNNs) or temporal order (unlike RNNs).
- **Parameter-heavy on raw high-dimensional data:** using a plain MLP directly on raw image pixels requires an enormous number of parameters and ignores spatial structure — this motivates CNNs.

- **Role in Generative AI:** MLturns/dense layers are used inside almost every architecture (e.g., as the final classification head, or inside encoder/decoder blocks of autoencoders and VAEs).

1.2 Convolutional Neural Networks (CNNs)

DEFINITION — Convolutional Neural Network

A specialized neural network architecture designed primarily for **grid-structured data** such as images, that uses **convolutional layers** to automatically and efficiently learn spatial hierarchies of features (edges → textures → parts → objects) while sharing parameters across spatial locations.

Core building blocks:

- **Convolutional Layer:** Slides small learnable filters/kernels (e.g., 3×3) across the input, computing dot products to produce **feature maps** that highlight patterns like edges or textures. Key idea: **parameter sharing** — the same filter is reused at every spatial location, drastically reducing parameters compared to a fully connected layer.
- **Stride:** the step size by which the filter moves across the input; larger stride reduces the output feature map size.
- **Padding:** adding extra pixels (usually zeros) around the input border so the output size can be controlled ('same' padding preserves size, 'valid' padding shrinks it).
- **Activation Function (usually ReLU):** applied after each convolution to introduce non-linearity.
- **Pooling Layer (e.g., Max Pooling, Average Pooling):** down-samples feature maps, reducing spatial dimensions while retaining the most important information, and providing a degree of translation invariance.
- **Fully Connected Layer(s):** typically placed at the end of the network to combine extracted features for the final classification/regression output.
- **Flatten Layer:** converts the final multi-dimensional feature maps into a 1-D vector before feeding into dense layers.

Why CNNs work well for images (key exam points):

- **Local connectivity:** each neuron only looks at a small local region (its receptive field) instead of the entire image, matching the local nature of visual patterns.
- **Parameter sharing:** drastically fewer parameters than a fully connected network on the same input, reducing overfitting and computation.
- **Translation invariance:** a learned filter can detect a feature (e.g., an edge) regardless of where it appears in the image, especially after pooling.
- **Hierarchical feature learning:** early layers learn low-level features (edges, colors, corners); deeper layers combine these into higher-level features (shapes, object parts, whole objects).

Role in Generative AI:

- CNNs (and their transposed/'deconvolution' counterparts) form the encoder and decoder/generator backbones of most image-based generative models — e.g., the Discriminator in DCGAN is a CNN classifier, and the Generator uses transposed convolutions to upsample noise into an image.
- Convolutional encoder/decoder pairs are the standard choice for building autoencoders and VAEs on image data.
- The U-Net (a CNN with an encoder-decoder + skip-connection design) is the standard backbone for the denoising network in Diffusion Models.

1.3 Recurrent Neural Networks (RNNs) and Variants

DEFINITION — Recurrent Neural Network

A neural network architecture designed for **sequential data** (text, time series, audio), which maintains a **hidden state** that is updated at every time step, allowing information from previous steps to influence the processing of the current step — effectively giving the network 'memory'.

Core recurrence relation:

$$h_t = f(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b)$$

where x_t is the input at time step t , h_{t-1} is the hidden state carried over from the previous time step, W_{hh} and W_{xh} are shared weight matrices (the same weights are reused at every time step), and h_t is the new hidden state. The output y_t at each step is typically computed as a function of h_t .

Training: Backpropagation Through Time (BPTT):

RNNs are trained by unrolling the network across all time steps and applying backpropagation across this unrolled graph — called Backpropagation Through Time. Because the same weights are reused at every step, gradients must be accumulated across the whole sequence.

Key limitation — Vanishing/Exploding Gradients:

- During BPTT, gradients are repeatedly multiplied across many time steps; if these repeated factors are consistently less than 1, gradients shrink toward zero (**vanishing gradients**), so the network fails to learn long-range dependencies.
- If the repeated factors are consistently greater than 1, gradients grow uncontrollably (**exploding gradients**), destabilizing training (often mitigated using gradient clipping).
- This limitation of plain/'vanilla' RNNs directly motivated the design of gated architectures — LSTM and GRU.

1.3.1 Long Short-Term Memory (LSTM)

An RNN variant introduced by Hochreiter & Schmidhuber (1997) that adds a separate **cell state** (a kind of conveyor belt of memory) and three learned **gates** that regulate the flow of information, allowing the network to retain or forget information over long sequences:

- **Forget Gate:** decides what proportion of the previous cell state to discard.
- **Input Gate:** decides what new information from the current input to add to the cell state.
- **Output Gate:** decides what part of the (updated) cell state should be exposed as the hidden state/output at this time step.
- Each gate is a small neural layer (typically Sigmoid-activated, outputting values between 0 and 1) that acts as a learned 'valve' controlling information flow.

1.3.2 Gated Recurrent Unit (GRU)

A simplified alternative to LSTM (Cho et al., 2014) that combines the forget and input gates into a single **update gate**, and adds a **reset gate** — resulting in fewer parameters and often faster training, while achieving comparable performance to LSTM on many tasks.

Role in Generative AI:

- RNN/LSTM/GRU architectures were the standard choice for sequence generation tasks (text generation, music generation) before Transformers became dominant.
- They illustrate the core problem that attention mechanisms and Transformers were later designed to solve — modelling long-range dependencies more effectively than recurrence allows.
- Still relevant in resource-constrained settings, streaming/online applications, and as a conceptual foundation for understanding sequence modelling.

1.4 Comparative Summary of Architectures

Aspect	Feedforward (MLP)	CNN	RNN (incl. LSTM/GRU)
Best suited for	Tabular / generic vector data	Grid data — images, spatial data	Sequential data — text, time series, audio
Key structural idea	Fully connected dense layers	Local receptive fields + shared filters (convolution) + pooling	Recurrent hidden state carried across time steps
Handles order/position?	No	Yes (spatial)	Yes (temporal)
Parameter efficiency	Low (dense connections)	High (parameter sharing across space)	High (parameter sharing across time)
Key limitation	Ignores structure in input; parameter-heavy on large inputs	Fixed input size typical; limited direct modelling of long sequences	Vanishing/exploding gradients on long sequences (mitigated by LSTM/GRU)
Typical role in Generative AI	Classification heads, dense blocks inside larger models	Encoder/Decoder/Generator/Discriminator backbones for image data; U-Net in diffusion	Sequence generation (text/music) prior to Transformer dominance

★ Exam Key Points — Section 1

- FNN/MLP: one-directional flow, fully connected, $h = f(Wx + b)$; trained via forward pass + backpropagation.
- CNN: convolution + pooling + shared filters give local connectivity, parameter efficiency, and translation invariance — ideal for images.
- RNN: maintains a hidden state $h_t = f(W_{hh} h_{t-1} + W_{xh} x_t + b)$; trained via Backpropagation Through Time (BPTT); suffers from vanishing/exploding gradients.
- LSTM (forget/input/output gates) and GRU (update/reset gates) were designed specifically to fix the vanishing gradient problem of vanilla RNNs.
- All three architectures reappear as building blocks inside autoencoders, VAEs, GANs, and Diffusion Models — this section is foundational for the rest of the course.

2. Review of Autoencoders: Vanilla and Sparse Autoencoders

2.1 What Is an Autoencoder?

DEFINITION — Autoencoder

An **unsupervised** neural network trained to reconstruct its own input at the output, by first compressing the input into a lower-dimensional internal representation (the **latent code / bottleneck**) and then reconstructing the original input from that compressed representation. The network learns purely from the data itself — the 'label' for training is simply the input x itself (a self-supervised setup).

General architecture — three parts:

- **Encoder, $f(x)$:** Compresses the input x into a lower-dimensional latent representation $z = f(x)$. Usually implemented with progressively narrowing dense or convolutional layers.
- **Bottleneck / Latent Code, z :** The compressed representation at the narrowest point of the network; forces the network to learn only the most important, informative features of the data.
- **Decoder, $g(z)$:** Reconstructs an approximation of the original input from the latent code: $x_{\hat{}} = g(z) = g(f(x))$. Usually mirrors the encoder architecture in reverse (progressively widening layers).

$$x_{\hat{}} = g(f(x)) \text{ — } Loss = Distance(x, x_{\hat{}})$$

Why the bottleneck is essential:

If the latent dimension were equal to (or larger than) the input dimension with no other constraint, the network could trivially learn an identity mapping (simply copy the input through) without learning anything useful. The bottleneck forces the network to discover a compressed, meaningful representation that captures the most salient structure/patterns in the data.

Typical uses of autoencoders:

- **Dimensionality reduction** — a non-linear alternative to PCA.
- **Denoising** — removing noise from corrupted input.
- **Anomaly detection** — inputs that are poorly reconstructed (high reconstruction error) are flagged as anomalies, since the network was only trained on 'normal' data.
- **Pretraining / feature learning** — learned latent features can be reused for downstream supervised tasks.
- **Foundation for generative modelling** — the Variational Autoencoder (Section 3) builds directly on this encoder-decoder idea, adding probability theory to make it generative.

2.2 Vanilla (Standard) Autoencoder

DEFINITION — Vanilla Autoencoder

The most basic form of autoencoder: a plain encoder-decoder network with a bottleneck layer smaller than the input dimension (an **undercomplete** architecture), trained purely to minimize reconstruction error, with no additional regularization or probabilistic constraint on the latent space.

Key characteristics:

- **Undercomplete design:** $\dim(z) < \dim(x)$ — this dimensionality reduction alone is the main mechanism forcing useful feature learning.
- **Deterministic mapping:** for a given input x , the encoder always produces the exact same latent vector z (no randomness/sampling involved) — this is a key contrast with VAEs (Section 3).
- **Loss function:** typically Mean Squared Error (MSE) for continuous data (e.g., natural images) or Binary Cross-Entropy for data in the $[0,1]$ range (e.g., normalized pixel intensities or binary data).
- **No structured/continuous latent space guarantee:** the latent space of a vanilla autoencoder can have 'gaps' or discontinuities — sampling a random point in latent space and decoding it does not reliably produce a realistic output. This is precisely why vanilla autoencoders are **not** good generative models, even though they share the encoder-decoder structure.

Vanilla Autoencoder loss (reconstruction only):

$$L(x, x_{\hat{}}) = \|x - x_{\hat{}}\|^2 \text{ (Mean Squared Error) or } L = -\sum [x \cdot \log(x_{\hat{}}) + (1-x) \cdot \log(1-x_{\hat{}})] \text{ (Binary Cross-Entropy)}$$

2.3 Sparse Autoencoder

DEFINITION — Sparse Autoencoder

An autoencoder that adds a **sparsity constraint/penalty** on the activations of the hidden (latent) layer, encouraging most latent neurons to be inactive (close to zero) for any given input, even when the hidden layer is **overcomplete** (i.e., has more units than the input dimension).

Unlike the vanilla autoencoder, which relies purely on a narrow bottleneck to force useful compression, a sparse autoencoder can use a wide (even overcomplete) hidden layer, but forces most neurons to stay inactive for each input via an explicit sparsity penalty added to the loss — encouraging the network to learn a small number of highly specialized, meaningful feature detectors rather than relying purely on dimensionality reduction.

How sparsity is enforced — two common approaches:

- **L1 Regularization on activations:** add a penalty term equal to the sum of absolute values of the hidden activations, $\lambda \cdot \sum |z_j|$, encouraging many activations to shrink to exactly zero.
- **KL-Divergence Sparsity Penalty:** define a target (small) average activation p (e.g., $p = 0.05$), compute the actual average activation $p_{\hat{}}_j$ of each hidden neuron j over the training batch, and penalize the KL divergence between the Bernoulli distributions $\text{Bernoulli}(p)$ and $\text{Bernoulli}(p_{\hat{}}_j)$, summed over all hidden units — this directly pushes each neuron's average activation toward the small target p .

$$L_{\text{sparse}} = L_{\text{reconstruction}} + \beta \cdot \sum_j \text{KL}(p \parallel p_{\hat{}}_j)$$

where β controls the strength of the sparsity penalty relative to the reconstruction loss, and $\text{KL}(p \parallel p_{\hat{}}_j) = p \cdot \log(p/p_{\hat{}}_j) + (1-p) \cdot \log((1-p)/(1-p_{\hat{}}_j))$ is the KL divergence between two Bernoulli distributions (this is a direct, concrete use case of KL Divergence — the same core concept revisited more generally in Section 4.2).

Why sparsity is useful:

- Encourages the network to learn more interpretable, disentangled, and specialized features (each neuron tends to 'fire' only for specific patterns).
- Prevents the trivial identity-mapping problem even when the hidden layer is large/overcomplete.

- Often produces features that transfer well to downstream supervised tasks.

2.4 Other Autoencoder Variants (for completeness)

Variant	Core Idea	Purpose
Undercomplete Autoencoder	Bottleneck dimension < input dimension (the 'vanilla' design)	Force compact, meaningful representation via dimensionality reduction
Overcomplete Autoencoder	Bottleneck dimension $\geq$ input dimension	Needs an extra constraint (like sparsity) to avoid learning a trivial identity mapping
Sparse Autoencoder	Overcomplete + sparsity penalty (L1 or KL-divergence) on activations	Learn specialized, interpretable feature detectors
Denoising Autoencoder (DAE)	Input is intentionally corrupted with noise; network must reconstruct the clean original	Learn robust features and prevent trivial copying
Contractive Autoencoder (CAE)	Adds a penalty on the Jacobian (sensitivity) of the encoder output w.r.t. the input	Learn representations robust to small input perturbations
Variational Autoencoder (VAE)	Encoder outputs a probability distribution (not a point); adds a KL regularization term	Enables generation of new data by sampling the latent space (covered in detail in Section 3)

★ Exam Key Points — Section 2

- Autoencoder = Encoder + Bottleneck (latent code) + Decoder, trained to minimize reconstruction loss $\|x - \hat{x}\|^2$.
- Vanilla Autoencoder: undercomplete, deterministic latent code, no probabilistic constraint — NOT reliably generative because its latent space has no guaranteed structure/continuity.
- Sparse Autoencoder: can be overcomplete; adds an L1 or KL-divergence sparsity penalty so only a few neurons activate per input, forcing specialized feature learning.
- Know the KL-divergence sparsity formula: $L = L_{\text{reconstruction}} + \beta \cdot \sum \text{KL}(p_{\text{enc}} \| p_{\text{dec}})$ — this is a concrete precursor to the KL divergence used in VAEs.
- Denoising and Contractive Autoencoders are the two other commonly asked variants — know their one-line core idea each.

3. Variational Autoencoders (VAEs): Latent Variable Representation and Reconstruction

3.1 From Deterministic to Probabilistic Autoencoders

As established in Section 2, a vanilla autoencoder maps each input x to a single fixed point z in latent space (a deterministic mapping), and its latent space has no guarantee of being smooth, continuous, or well-organized — so it cannot reliably generate new data by sampling. The **Variational Autoencoder (VAE)**, introduced by Kingma & Welling (2013), solves this by turning the encoder into a **probabilistic** mapping: instead of outputting a single latent point, the encoder outputs the **parameters of a probability distribution** over the latent space.

DEFINITION — Variational Autoencoder (VAE)

A generative model that learns a probabilistic encoder $q_{\phi}(z|x)$ and a probabilistic decoder $p_{\theta}(x|z)$ jointly, such that the latent space is regularized to follow a known, simple prior distribution (usually a standard Gaussian), enabling new realistic data to be generated by sampling z from the prior and decoding it.

3.2 Latent Variable Representation

The central idea of a VAE is that observed data x is assumed to be generated from some underlying, unobserved (**latent**) random variable z , through a generative process:

- **Step 1 — Sample a latent vector:** $z \sim p(z)$, where $p(z)$ is a simple prior distribution, almost always chosen as a standard multivariate Gaussian, $p(z) = N(0, I)$ (see Unit 1, Section 3.3 for the multivariate Gaussian).
- **Step 2 — Generate data conditioned on z :** $x \sim p_{\theta}(x|z)$, where $p_{\theta}(x|z)$ is the decoder, a neural network with parameters θ that defines a distribution over data given the latent code.

The Inference Problem and the Approximate Posterior:

To train this model we would ideally like to compute the true posterior $p(z|x)$ — 'given this data point, what latent value likely produced it?' — but this posterior is **intractable** to compute directly (it requires integrating over all possible z). The VAE solves this by introducing an **encoder network $q_{\phi}(z|x)$** , with its own parameters ϕ , that learns to **approximate** the true posterior. This is exactly why the method is called 'variational' — it uses techniques from variational inference to approximate an intractable posterior with a tractable, learnable distribution.

What the encoder actually outputs:

- For each input x , the encoder network outputs two vectors: a **mean vector $\mu(x)$** and a **(log-)variance vector $\sigma^2(x)$** (or $\log \sigma^2(x)$, for numerical stability), which together fully define a diagonal Gaussian distribution over the latent variable: $q_{\phi}(z|x) = N(z; \mu(x), \sigma^2(x) \cdot I)$.
- This is a direct application of the diagonal-covariance multivariate Gaussian introduced in Unit 1, Section 3.3 — each latent dimension is treated as an independent Gaussian with its own learned mean and variance.
- Each input x therefore maps not to one point but to a small 'cloud'/region of probable latent codes — this is what gives the VAE latent space its smoothness and continuity.

Why the latent space becomes smooth and meaningful:

- The KL-divergence term in the VAE loss (Section 4.2) continuously pulls every $q_\phi(z|x)$ toward the same shared prior $N(0, I)$.
- Because many different inputs' latent distributions are all pulled toward — and overlap within — the same region, the latent space becomes densely and smoothly filled, with no 'holes'.
- As a direct consequence, interpolating between two latent points, or sampling any random point $z \sim N(0, I)$, reliably decodes into a plausible, realistic output — this is the property a vanilla autoencoder's latent space does NOT reliably have.

3.3 The Reparameterization Trick

A critical implementation detail: we need to sample z from $q_\phi(z|x) = N(\mu, \sigma^2)$ during training, but the sampling operation itself is **random** and therefore **not differentiable** — gradients cannot be backpropagated through a random sampling node directly, which would make it impossible to train the encoder's μ and σ parameters end-to-end with standard backpropagation.

The Reparameterization Trick solves this as follows:

$$z = \mu(x) + \sigma(x) \blacksquare \epsilon, \text{ where } \epsilon \sim N(0, I)$$

- The randomness is isolated into an auxiliary noise variable ϵ , sampled independently from a fixed standard normal distribution $N(0, I)$ — this step has no learnable parameters, so no gradient needs to flow through it.
- z is now expressed as a **deterministic, differentiable function** of the learnable parameters $\mu(x)$ and $\sigma(x)$, plus the independent noise ϵ .
- Gradients of the loss with respect to μ and σ can now be computed and backpropagated normally through the encoder, enabling standard end-to-end training via stochastic gradient descent.
- $\blacksquare$ denotes element-wise (Hadamard) multiplication, since μ , σ , and ϵ are all vectors matching the latent dimensionality.

3.4 Reconstruction in VAEs

The full forward pass of a VAE, step by step:

1. **Encode:** pass input x through the encoder network to obtain $\mu(x)$ and $\sigma(x)$ — the parameters of $q_\phi(z|x)$.
2. **Sample:** draw $\epsilon \sim N(0, I)$ and compute the latent vector using the reparameterization trick: $z = \mu(x) + \sigma(x) \blacksquare \epsilon$.
3. **Decode:** pass z through the decoder network $p_\theta(x|z)$ to produce a reconstruction $x^\dagger$ (or, more precisely, the parameters of a distribution over x , such as a Bernoulli mean for binary pixel data, or a Gaussian mean for continuous data).
4. **Compute Loss:** compare $x^\dagger$ to the original x using a reconstruction loss term, and simultaneously penalize how far $q_\phi(z|x)$ is from the prior $p(z)$ using the KL divergence term (full loss detailed in Section 4.4).
5. **Backpropagate and Update:** update both encoder parameters ϕ and decoder parameters θ jointly using gradient descent.

Two distinct modes of using a trained VAE:

Mode	Process	Purpose
Reconstruction	Encode a real input $x \rightarrow$ sample z from $q_\phi(z x) \rightarrow$ decode z back to x	Check how well the model has learned to represent/compress the specific input data (used mainly during training/evaluation)
Generation (Sampling)	Skip the encoder entirely $\rightarrow$ sample z directly from the prior $N(0, I) \rightarrow$ decode z	Create brand-new, previously unseen synthetic data samples (the true generative use case)
Interpolation	Encode two real inputs $\rightarrow$ linearly interpolate between their latent codes $\rightarrow$ decode intermediate points	Produce smooth morphs between two data points, showcasing the continuity of the learned latent space

3.5 VAE vs Vanilla Autoencoder — Direct Comparison

Aspect	Vanilla Autoencoder	Variational Autoencoder (VAE)
Encoder output	A single fixed latent vector z	Parameters (μ, σ^2) of a distribution over z
Latent space structure	No guaranteed structure; may have gaps/discontinuities	Smooth, continuous, densely filled (regularized toward $N(0, I)$)
Loss function	Reconstruction loss only	Reconstruction loss + KL Divergence regularization (ELBO)
Can generate new data?	Not reliably (sampling random latent points often gives unrealistic output)	Yes — sampling $z \sim N(0, I)$ and decoding reliably gives realistic new samples
Underlying approach	Deterministic dimensionality reduction	Probabilistic / variational inference
Output sharpness	Sharper (but not generative)	Often slightly blurrier due to the KL regularization/reconstruction trade-off

★ Exam Key Points — Section 3

- VAE = probabilistic encoder $q_\phi(z|x)$ + probabilistic decoder $p_\theta(x|z)$, trained via variational inference to approximate the intractable true posterior $p(z|x)$.
- The encoder outputs $\mu(x)$ and $\sigma(x)$, defining a diagonal Gaussian $q_\phi(z|x) = N(\mu, \sigma^2 I)$ — a direct application of the multivariate Gaussian.
- Reparameterization trick: $z = \mu + \sigma \epsilon$, $\epsilon \sim N(0, I)$ — makes sampling differentiable so backpropagation works end-to-end.
- Generation at test time bypasses the encoder entirely: sample $z \sim N(0, I)$ directly, then decode.
- The KL term is what makes the latent space smooth/continuous — this is the single biggest structural difference from a vanilla autoencoder.

4. Loss Functions: Cross-Entropy, KL Divergence, and Reconstruction Loss

This section consolidates the three core loss-function concepts used throughout autoencoders, VAEs, and generative modelling in general, building on the probability foundations from Unit 1.

4.1 Cross-Entropy Loss

DEFINITION — Cross-Entropy

A measure from information theory of the **difference between two probability distributions** — the true distribution p and a predicted/model distribution q . In machine learning it is used as a loss function that penalizes a model for how far its predicted probability distribution is from the true (target) distribution; the loss is minimized when the predicted distribution exactly matches the true one.

General formula:

$$H(p, q) = - \sum_i p(i) \cdot \log(q(i))$$

Binary Cross-Entropy (BCE) — used for binary classification or binary/normalized [0,1] pixel data:

$$L_{BCE} = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

where y is the true label/value (0 or 1) and $\hat{y}$ is the model's predicted probability. BCE is the standard reconstruction loss choice in autoencoders/VAEs when input pixels are normalized to the range [0, 1] and interpreted as Bernoulli probabilities.

Categorical Cross-Entropy — used for multi-class classification:

$$L_{CCE} = - \sum_c y_c \cdot \log(\hat{y}_c)$$

where the sum runs over all classes c , y_c is 1 for the true class and 0 otherwise (one-hot encoding), and $\hat{y}_c$ is the predicted probability for class c (usually produced via a Softmax output layer).

Key properties and intuition:

- Cross-entropy heavily penalizes **confident but wrong** predictions — if the model predicts a probability close to 0 for the true class, $-\log(\hat{y})$ becomes very large.
- Cross-entropy is closely related to (but not identical to) KL Divergence: $H(p, q) = H(p) + D_{KL}(p \parallel q)$, where $H(p)$ is the entropy of the true distribution. When p is a fixed one-hot ground-truth label, $H(p) = 0$, so minimizing cross-entropy is equivalent to minimizing the KL divergence between the true and predicted distributions.
- In autoencoders/VAEs, cross-entropy is one of the two standard choices (along with MSE) for the reconstruction loss term, applicable specifically when the data can be interpreted as probabilities/binary values.

4.2 KL Divergence (Kullback–Leibler Divergence)

DEFINITION — KL Divergence

A measure of how one probability distribution q diverges from a second, reference probability distribution p . It quantifies the 'extra information' or 'inefficiency' incurred when using distribution q to approximate the true distribution p . **Important:** KL Divergence is NOT a true distance metric — it is asymmetric, meaning $D_{KL}(p \parallel q) \neq D_{KL}(q \parallel p)$ in general, and it does not satisfy the triangle inequality.

General formula (discrete case):

$$D_{KL}(p \parallel q) = \sum_i p(i) \cdot \log(p(i) / q(i))$$

Continuous case:

$$D_{KL}(p \parallel q) = \int p(x) \cdot \log(p(x) / q(x)) dx$$

Key properties (frequently tested):

- **Non-negativity:** $D_{KL}(p \parallel q) \geq 0$ for all distributions p and q .
- **Zero if and only if identical:** $D_{KL}(p \parallel q) = 0$ if and only if $p = q$ everywhere.
- **Asymmetry:** $D_{KL}(p \parallel q) \neq D_{KL}(q \parallel p)$ in general — this is why it is called a 'divergence' rather than a 'distance'.
- **Not a metric:** does not satisfy the triangle inequality, unlike true distance measures such as Euclidean distance.

Closed-form KL Divergence between two Gaussians (essential formula for VAEs):

Since the VAE's approximate posterior $q_{\phi}(z|x) = N(\mu, \sigma^2)$ and prior $p(z) = N(0, I)$ are both Gaussian, the KL divergence between them has a convenient closed-form solution (no need for numerical integration), computed independently per latent dimension j and summed:

$$D_{KL}(N(\mu, \sigma^2) \parallel N(0, 1)) = - (1/2) \cdot \sum_j [1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2]$$

How to read this formula intuitively:

- This term is minimized (equals 0) exactly when $\mu_j = 0$ and $\sigma_j^2 = 1$ for every latent dimension j — i.e., when the encoder's output distribution exactly matches the standard normal prior.
- If μ drifts away from 0, the μ_j^2 term grows, penalizing the encoder for moving the latent distribution's centre away from the prior's centre.
- If σ^2 drifts away from 1 (either too large or too small), the $(1 + \log \sigma^2 - \sigma^2)$ portion penalizes the encoder, pulling the spread of each latent dimension back toward matching the prior's spread.
- This closed-form expression is what makes it computationally cheap and exact to include the KL term directly in the VAE's training loss at every step.

Role of KL Divergence in a VAE (recap and consolidation):

- Acts as a **regularizer** on the latent space, pulling every input's latent distribution toward the shared prior $N(0, I)$.
- This regularization is exactly what gives the VAE's latent space its smoothness/continuity, enabling reliable generation by sampling from the prior (see Section 3.2).
- There is a direct trade-off: too much KL weight over-regularizes the latent space causing poor reconstructions (this is the idea behind the β -VAE variant, which scales the KL term by a factor β to control this trade-off); too little KL weight leaves the latent space unstructured, closer to a vanilla autoencoder.

4.3 Reconstruction Loss

DEFINITION — Reconstruction Loss

The component of the loss function that measures how closely the decoder's output $\hat{x}$ matches the original input x — i.e., how well the model can 'reconstruct' the data after compressing it through the bottleneck/latent code. It is the term shared by ALL autoencoder-family models (vanilla, sparse, denoising, and VAE).

The two standard choices, and when to use each:

Loss Type	Formula	When Used
Mean Squared Error (MSE)	$L = (1/n) \cdot \sum (x_i - \hat{x}_i)^2$	Continuous-valued data (e.g., natural images with real-valued/unbounded pixel intensities); assumes a Gaussian output distribution $p(x z)$
Binary Cross-Entropy (BCE)	$L = -\sum [x_i \cdot \log(\hat{x}_i) + (1-x_i) \cdot \log(1-\hat{x}_i)]$	Data normalized to [0, 1] and interpreted as probabilities (e.g., binarized/normalized images such as MNIST); assumes a Bernoulli output distribution $p(x z)$

Probabilistic interpretation (important conceptual link):

In a VAE, the reconstruction term is technically the expected log-likelihood $E_{q(z|x)}[\log p_{\theta}(x|z)]$ — i.e., how probable the true input x is, under the distribution that the decoder defines given the sampled latent z . Choosing MSE as the reconstruction loss is mathematically equivalent to assuming the decoder output follows a Gaussian distribution with fixed variance; choosing Binary Cross-Entropy is equivalent to assuming a Bernoulli output distribution. This is why the 'choice' of reconstruction loss is really a choice of an assumed output probability distribution, not an arbitrary formula.

Practical notes:

- MSE tends to produce blurrier reconstructions/generations because it penalizes large pixel-wise errors heavily and encourages the model to output an 'average' of plausible values.
- BCE requires inputs (and outputs) to be scaled/normalized into the [0, 1] range beforehand.
- The reconstruction loss alone (without any regularization) is exactly the vanilla/sparse autoencoder's full loss; in a VAE it becomes just one of the two terms in the total loss.

4.4 Putting It Together — The VAE Total Loss (ELBO)

The complete VAE training objective combines Reconstruction Loss and KL Divergence into a single quantity called the **Evidence Lower Bound (ELBO)**, which the model tries to **maximize** (equivalently, the negative ELBO is minimized as the loss):

$$ELBO(x) = E_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \parallel p(z))$$

$$Total\ VAE\ Loss = Reconstruction\ Loss\ (MSE\ or\ BCE) + KL\ Divergence\ Term\ (minimized)$$

Why this combination is called a 'lower bound':

The true objective we would like to maximize is the log-likelihood of the data, $\log p_{\theta}(x)$, but this quantity is intractable to compute directly (it requires integrating over all possible latent values z). It can be mathematically shown that $\text{ELBO}(x) \leq \log p_{\theta}(x)$ always holds — the ELBO is a provable lower bound on the true log-likelihood. Therefore, maximizing the ELBO indirectly pushes the true data log-likelihood upward as well, giving a tractable, well-justified training objective.

Interpreting the two-term trade-off (a very common exam question):

- **Reconstruction term** pulls the model toward accurately reproducing each specific training input — encourages fidelity/accuracy.
- **KL term** pulls every input's latent distribution toward the shared standard Gaussian prior — encourages a smooth, well-organized, generative-friendly latent space.
- These two terms are in **tension**: perfect reconstruction alone (ignoring KL) would push the model back toward vanilla-autoencoder-like behavior with an unstructured latent space; too much KL weight alone would ignore the actual input and just match the prior, producing poor/generic reconstructions.
- The β -VAE variant makes this trade-off explicit and tunable by adding a weighting hyperparameter β in front of the KL term: $\text{Loss} = \text{Reconstruction} + \beta \cdot D_{\text{KL}}$ — increasing β produces a more disentangled/structured latent space at the cost of reconstruction quality.

★ Exam Key Points — Section 4

- Cross-Entropy: $H(p, q) = -\sum p(i) \log q(i)$; Binary Cross-Entropy $L = -[y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y})]$ is used for $[0,1]$ -normalized reconstruction targets.
- KL Divergence: $D_{\text{KL}}(p \parallel q) = \sum p(i) \log(p(i)/q(i))$; always ≥ 0 , asymmetric, and equals 0 only when $p = q$.
- Closed-form Gaussian KL used in VAEs: $D_{\text{KL}} = -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ — minimized exactly when $\mu=0$, $\sigma^2=1$.
- Reconstruction Loss = MSE (continuous data, Gaussian assumption) or BCE (data in $[0,1]$, Bernoulli assumption).
- VAE Total Loss (negative ELBO) = Reconstruction Loss + KL Divergence — a direct trade-off between accurate reconstruction and a smooth, generative-friendly latent space; β -VAE makes this trade-off tunable.

Unit 2 — Quick Revision Summary

Use this page the night before the exam to rapidly recall every sub-topic in Unit 2.

1. Neural Network Architectures

- Feedforward (MLP): one-directional, fully connected, $h = f(Wx+b)$; trained via forward pass + backprop.
- CNN: convolution (shared filters) + pooling; local connectivity, parameter sharing, translation invariance — ideal for images.
- RNN: hidden state $h_t = f(W_{hh} h_{t-1} + W_{xh} x_t + b)$; trained via BPTT; suffers vanishing/exploding gradients; LSTM (3 gates) and GRU (2 gates) fix this.

2. Autoencoders

- Autoencoder = Encoder + Bottleneck + Decoder; trained to minimize reconstruction loss $x_{\text{recon}} = g(f(x))$.
- Vanilla: undercomplete, deterministic, latent space not guaranteed smooth — not reliably generative.
- Sparse: can be overcomplete + sparsity penalty (L1 or KL-divergence on activations) forces specialized features.
- Other variants: Denoising (reconstruct clean input from corrupted input), Contractive (penalize Jacobian sensitivity).

3. Variational Autoencoders

- VAE encoder outputs $\mu(x), \sigma(x) \rightarrow$ defines $q_{\phi}(z|x) = N(\mu, \sigma^2 I)$, approximating the intractable true posterior $p(z|x)$.
- Reparameterization trick: $z = \mu + \sigma \epsilon, \epsilon \sim N(0, I)$ — makes sampling differentiable for backpropagation.
- Generation: sample $z \sim N(0, I)$ directly (skip encoder) $\rightarrow$ decode $\rightarrow$ new realistic sample.
- KL term regularizes latent space toward $N(0, I)$, making it smooth/continuous — the key difference from vanilla autoencoders.

4. Loss Functions

- Cross-Entropy: $H(p, q) = -\sum p(i) \log q(i)$; BCE for $[0, 1]$ data, Categorical CE for multi-class.
- KL Divergence: $D_{\text{KL}}(p \parallel q) = \sum p(i) \log(p(i)/q(i))$; ≥ 0 , asymmetric, not a true metric.
- Closed-form Gaussian KL for VAEs: $-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$.
- Reconstruction Loss: MSE (continuous/Gaussian assumption) or BCE (bounded/Bernoulli assumption).
- VAE Total Loss (negative ELBO) = Reconstruction Loss + KL Divergence — β -VAE tunes this trade-off via a β weight on the KL term.

End of Unit 2 Detailed Notes.